

Delegate: Coalition Proof Incentivized Outsourced Computation with Smart Contracts^{*}

Osman Biçer¹ and Alptekin Küpçü²[0000–0003–2099–2206]

¹ University of Basel, Switzerland

² Koç University, Türkiye

Abstract. In incentivized outsourced computation, an outsourcer hands a computation to multiple contractors, and in return expects the same computation output from each. The state-of-the-art proposals succeed in incentivizing all contractors for honest computation by setting this behavior as the unique Nash equilibrium of the system. Yet, they are either limited to the two-contractor case or the multi-contractor case without security proof against fully collaborating coalitions. Further, employing a smart contract for managing submissions from the contractors has been a non-trivial task due to the copy attack. In this work, we target this issue and propose a smart contract based multi-contractor incentivized outsourced computation protocol **Delegate**, which is provably secure against coalitions. We further mitigate the copy attack by a universally composable response submission protocol based on commitments, in contrast to previous solutions which only disincentivize it.

Keywords: outsourced computation, smart contract, game theory

1 Introduction

A client can improve its computation capability and memory capacity by outsourcing computation to cloud providers or to volunteer nodes. To securely and dependably outsource a heavy computation to third parties, there has been a long line of approaches ranging from the ones based on cryptography [38, 62, 72, 42, 65] to those based on game theory [7, 54, 28, 49, 5, 27, 55]. Among the latter, game-theory based *incentivized outsourced computation*, where the same computational task is outsourced to more than one contractor, is an interesting arising subject studied in the two-party [7, 5, 55] and multi-party [54, 9] settings. In this approach, a **boss** plans to outsource a computation on an input (while neither the computation nor the input being necessarily private), and is interested in receiving the **correct computation result**. If the boss runs the computation from scratch to verify the correctness of the computation, outsourcing it would not result in targeted efficiency improvement. Although specifically for NP problems there exist polynomial-time verification algorithms, we are interested in practical efficiency, which may not be achieved by high degree polynomial verification.

^{*} We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 119E088 and 123E462.

Regarding the multiple contractor setting, the probability of all the contractors being maliciously driven is lowered. The previous work [54] and its extension [9] provide a mechanism where the *honest contractors* help the outsourcer catch and punish *lazy contractors*. The former achieves all contractors diligently executing the computation as the Nash equilibrium, while the latter fine-tunes the mechanism for ensuring security against a coalition with up to all involved contractors except one. However, none of these works analyze the full collaboration of contractors for sharing the computation load.

A recent approach in incentivized outsourced computation has been using *smart contracts* [28, 49, 5, 27, 55]. Each party deposits some amount to the smart contract at the beginning of the protocol execution. The smart contract collects the results on behalf of the boss and returns to each party the corresponding outcome of the protocol, e.g., the computation result to the boss and a **reward** or a **fine** to each contractor. This way, the use of smart contracts allows a higher efficiency for the resource-restrained boss and protects a diligent contractor from the boss’s potential malicious actions, i.e., not paying the reward. However, none of these protocols are in the multi-contractor setting or provide any security guarantee against Sybils or coalitions.

Avizheh et al. proposed the *copy attack* in the smart contract setting [5]. As the contractors need to send the computation result publicly to the smart contract for verification, a copy attacker can copy a response previously submitted instead of executing the computation herself. Although [55] provides a “Match Check” protocol in the two-party setting to resolve this problem, there still exists a need for analyzing complicated outcomes that combine the copy attack with various other strategies and resolving it in the multi-party setting.

Overview of Our Techniques: In this work, we target to resolve the above-mentioned issues with incentivized outsourced computation. The setting that we consider is a combination of those of [54] and [55] such that a boss outsources the computation to $n > 1$ contractors and delegates the work of checking the final result to a smart contract. We model each contractor as a *bounded rational* adversary (inspired by [64]), which is rational by having the main interest to maximize its utility and is bounded to execute only a probabilistic polynomial time (PPT) interactive Turing machine (ITM).

At the start of the execution, the boss deposits to the smart contract (SC) a sufficient amount of currency for rewarding all the contractors. Each contractor deposits to the SC a sufficient amount of currency for the fine. Then, the contractors are given a deadline τ_{SC} for executing the computation and submitting their commitments to the computation results and the Merkle hash tree root of the computation steps. After τ_{SC} , they have another deadline τ_F to submit the openings of their commitments. We create a response submission protocol that is universally composable (UC) [18] based on non-interactive reusable-CRS (common reference string) commitments.

At τ_F , the SC gets activated and checks whether there exists a consensus in the submitted results from the contractors. If this is the case, then SC returns the result to the boss, gives back the deposit of fines to the contractors, and

distributes the rewards to the contractors. Otherwise, it runs the MismatchCheck protocol of [54] to determine the final result. Each MismatchCheck execution takes place among two contractors and SC, and assigns one of the contractors as the winner and the other one as the loser (never both are assigned as the winner or the loser). It is designed in a way that it will always assign a diligent contractor as the winner, and incentivizes participation.

To involve the used cryptographic protocols in the game theoretical analysis, regarding the Merkle hash tree, the bounded rational adversary may take any malicious action with negligible probability. Let the utility from this attack be a polynomial $poly_a(\lambda)$ of the security parameter λ if this attack is successful. Then, the expected utility due to this attack action is a negligible utility computed as $poly_a(\lambda) \cdot \epsilon$, where ϵ is the negligible probability of success.

Regarding the involvement of the UC-secure response submission protocol, we consider that each participant in the protocol is controlled by a player and the possible messages by the environment to that participant are essentially actions of each player. Although we do not assume the existence of the ideal functionality, any distinguishable outcome results in a breach for UC-security, therefore can only succeed with negligible probability, resulting in negligible utility.

We highlight that the equilibrium and coalition-proofness concepts in our game theoretical analysis are computational versions of the generic definitions. They allow a deviation to receive a higher utility only by a negligible function, compatible with bounded rational adversaries [51]. **Our Contributions are:**

- In Section 3, we provide a UC-secure [18] response submission protocol for smart-contract-assisted multi-party incentivized outsourced computation based on UC commitments [19, 23, 33, 35, 36]. Our protocol prevents the copy attack of [5], and eliminates the need for further analysis of strategies involving this attack, as long as the submitter of a response does not choose to leak it on purpose, which can be considered as forming a coalition. Being UC-secure, our proposal has strong security in scenarios where multiple protocols run concurrently, such as a blockchain-based distributed system.
- In Section 4, we propose our smart-contract-assisted n -party incentivized outsourced computation mechanism: **Delegate**. Compared to previous n -party protocols [54, 9], our proposed protocol incentivizes each contractor not only to execute the computation diligently but also to submit the correct computation output as soon as he can, and to help catch the cheating contractors. Further, our proposed n -party mechanism achieves a low fine-to-reward ratio f/r , which, according to [7], should be kept low for incentivization of contractors to join the system. This becomes a particularly important issue in a smart-contract-managed system, as each contractor is expected to deposit a fine at the beginning of the execution. More concretely, we achieve a fine-to-reward ratio of $\frac{f}{r} \approx \frac{1}{n-1}$ that is much lower than that of [54, 9], which was $1 \leq \frac{f}{r} \leq n-1$.
- In Section 5, we analyze our mechanism in the setting where contractors are allowed to form coalitions and to collaborate for sharing the computation load and the payment of fines and rewards. We show that in this setting, our

mechanism is $n - 1$ -repellent according to the ℓ -repellence definition of [9]. This shows that as long as all contractors in the system do not establish a coalition (i.e., there exists a separate party outside the coalition), the boss is guaranteed to receive the correct output since rational contractors will compute the task diligently.

Related Work:

Mechanism Design. In distributed systems and multi-party computation, mechanism design is used extensively with some notable works [45, 60, 1, 7, 34, 24, 2, 54]. There exist several proposed models such as the BAR model [4, 56], the Type model [32], and the rational protocol design (RPD) framework [37].

Regarding coalition proofness, previously, k -resiliency [1] has been the aim of a line of works [74, 2, 15, 46]. Yet, [9] has recently provided further ℓ -repellence and m -stability definitions for including the transferable utility assumption from game theory, and a tool to analyze those [71]. Some other models [56, 47] are based on simple coalitional abilities such as “cheap talk” [31].

Outsourced Computation. There exist great differences in outsourced computation approaches in terms of their provided guarantees, employed techniques, and efficiency. On one end, we have the ones based on fully homomorphic encryption [39]. Some other privacy preserving methods are based on secure multi-party computation [40, 8] and private function evaluation [10, 12]. Although they are designed for joint computation, some extensions have been developed for outsourcing specific computations [50, 59, 48].

If one considers the efficiency of the computation and correctness of the result to be more important than the privacy of the computation, other lines of work come into play. One line of work, called verifiable computation [38] (see [73] for a review), utilizes zero-knowledge proofs [43, 41] to achieve this target. Also, when the inputs of the computation are not private, proofs without zero-knowledge property can be utilized.

Some other outsourced computation approaches worth mentioning comprise the ones based on attribute-based encryption [62], probabilistically checkable proofs [72, 42], secure functional evaluation [65], and Byzantine agreement [4, 13]. There also exist some implemented ones [66, 67, 14, 61].

Some previous works [44, 69, 30] utilize “inner state hashes” to verify the computation result cheaply for the outsourcer. In fact, the technique utilized in this paper for verification is also an extension of [7, 54, 55, 9], by letting the smart contract to handle the verification task, as in [55].

Incentivized Outsourced Computation. The previous incentivized outsourced computation works that have the closest settings and goals to ours can be listed as [7, 54, 28, 49, 27, 55, 68]. Our setting differs from them as follows:

- [49, 27] are in a single-contractor setting, while [7, 28, 55] are in a two-contractor setting. Our work essentially differs from these works by outsourcing to *multiple* contractors.
- [54, 9] are in a multi-contractor setting as our work. They also outsource the same computation to multiple contractors and incentivize them to submit the correct responses. [9] further achieves security against coalitions. Yet, our

IOC scheme	Number n of Contractors	Smart Contract	Ratio f/r	Fast Submission	Cheater Hunt	Coalition/Sybil Proofness
[7]	2	N/A	~ 0	No incentive	N/A	N/A
[54]	≥ 2	N/A	$\sim (n-1)$	No incentive	Volunteered	Arrangeable as [9]
[55]	2	Yes	~ 0	No incentive	Forced	N/A
[9]	≥ 2	N/A	$\sim (n-1)$	No incentive	Volunteered	$n-1$, non-collaborating
Delegate	≥ 2	Yes	$\sim 1/(n-1)$	Incentivized	Incentivized	$n-1$, collaborating

Table (1) Detailed comparison of our incentivized outsourced computation scheme with the state-of-the-art in terms of our goals listed in Section 4. We highlight that the lower the ratio f/r is the more servers are incentivized to join as constructors [7]. Coalitions in [9] were not considered as collaborating for computation.

Scheme	Effectiveness	Offline Cost	Online Cost	Cost to SC
[55]	disincentivizes attack	no cost	Merkle tree proofs	Merkle tree verifications
π_{RS}	prevents attack	commitment	no cost	commitment verification

Table (2) Comparison of the copy attack countermeasures: our response submission protocol π_{RS} with the Match Check protocol of [55]. The costs are provided per contractor.

proposal further incentivizes submitting the correct and early computation output and helping catch cheaters, and we are in a smart-contract setting.

We compare our proposal with the existing mechanisms in Table 1. We acknowledge that these previous works do not target our listed goals, and it may be possible to further improve them to achieve some of the given features (e.g., it may be possible to improve [54, 9] by allowing the boss to employ a smart contract and to use our response submission protocol).

According to [7] a lower f/r ratio is better for incentivization of a server to join the system as a contractor. Therefore, it is considered as desirable. Although the ratios are not strictly enforced by any of the given protocols, we consider minimum possible ratios for each of them. It can be seen that the only limiting factors for the number of contractors in our mechanism are the number of available servers and the value of the reward $r > \text{cost}(d)$ as $n \cdot r$ plus gas costs needs to be higher than the value of the computation to the boss. Another line of work uses trusted execution environments [21, 75].

Copy attack. Proposed by Avizheh et al., the copy attack exploits the vulnerability on smart contract based incentivized outsourced computation schemes [5], as the involved contractors can just copy the first submitted response and obtain consensus of the results without actually executing the computation steps. Then, obtaining the correct result relies solely on the diligence of the first submitter. [55] disincentivizes the attack at the cost of a Match Check protocol by the contractors and the smart contract that takes place at the end of the computation. On the other hand, our UC secure response submission protocol π_{RS} prevents this attack completely without the need of any protocol at the end. We highlight that π_{RS} only takes as input the computation results (y, z) , whereas an execution of Match Check is also dependent on the function f that is computed. Therefore, the costs from π_{RS} is constant no matter what the num-

ber of the computation steps of f is. This contrasts with the costs from [55], which are asymptotically logarithmic with the number of the computation steps of f . Nevertheless, in cases of an implementation with the state-of-the art primitives, the Match Check may be less costly than π_{RS} , due to the availability of lighter weight hash algorithms compared to the public key operations required by UC-secure commitment schemes (see Table 2).

Simultaneously with our work, Avizheh et al. [6] created another smart contract based solution similarly in the universal composability (UC) model. Their main solution works for two contractors, and then they use $\binom{n}{2}$ instances of their two-contractor solution to achieve a multi-contractor solution. While techniques used (UC commitments) are similar, our analysis additionally includes collaborating coalition based game theoretic models (see Section 5). Another recent work specializes on Federated Learning [70].

2 Preliminaries and Subprotocols

Negligible Function. A function $n(\cdot) : \mathbb{N} \rightarrow \mathbb{R}$ is *negligible*, if for each positive polynomial $poly(\cdot)$ there exists an integer $N > 0$ such that for all $\lambda > N$ we have $|n(\lambda)| < \frac{1}{poly(\lambda)}$. If a negligible function takes as input the security parameter to output a probability or a utility, we may call it negligible probability or negligible utility, respectively.

Smart Contract. A smart contract (SC) is utilized for establishing binding contracts in cryptocurrencies (e.g., Ethereum [16]). SC is publicly executed by the underlying consensus protocol. It collects the submitted transactions and appends them to the blockchain. We assume SC as a trusted party just as in other smart contract based protocols [53, 3, 5, 55]. It can get activated at a set time by a trustless service such as Ethereum Alarm Clock.³

We define an n -player game by a set of players $N = (P_1, \dots, P_n)$, the m_i possible actions $\mathcal{A}_{P_i} = (a_{P_i,1}, \dots, a_{P_i,m_i})$ of each player $P_i \in N$, and a utility function $u_{P_i}(\cdot)$ of each player $P_i \in N$ that assigns a real value for each possible action profile. Each player targets to maximize her own utility. A strategy s_{P_i} of player P_i is a probability distribution over the possible actions of the player P_i . When we call an action a as a strategy a , we mean the strategy that assigns probability 1 to action a . Also, we denote by $s = (s_{P_1}, \dots, s_{P_n})$ a strategy profile composed of the strategies of all players. Further, we denote by $s_{-P_i} = s - \{s_{P_i}\}$ the strategy profile of all players excluding the player P_i . Given a strategy profile s , we denote by $u_i(s)$ the expected utility of the player P_i . We utilize the notion of *computational Nash equilibrium* defined as follows:

Definition 1 (Computational Nash equilibrium). A strategy set $s = (s_{P_1}, \dots, s_{P_n})$ is a computational Nash equilibrium if $\forall P_i \in N \ \forall s'_{P_i} \neq s_{P_i} \ u_{P_i}(s_N) \geq u_{P_i}(s'_i \cup s_{-i}) - n(\lambda)$ where $n(\cdot)$ is a negligible function and λ is the security parameter.

³ <https://github.com/ethereum-alarm-clock/ethereum-alarm-clock>

Transferable Utility. In many cooperative games, one can assume that players can simply and freely sign binding agreements for sharing the game outcome. The conventional name of this assumption is the transferable utility assumption [63]. This simplifies the analysis by defining a single total utility per coalition. More concretely, the utility of a coalition is defined as follows:

Definition 2 (Coalition Utility). *Let s be a strategy set for a game Π played by n players. Then, the utility $u_G(s)$ of a coalition G is defined as $u_G = \sum_{P_i \in G} u_{P_i}(s)$.*

This assumption is also useful in cases where a rational party joins a protocol with multiple identities, e.g., as in Sybil attack in peer-to-peer systems [29].

Definition 3 (Computationally Dominant Strategy). *A strategy s_{P_i} of a player P_i is its computationally dominant strategy if for all strategies $s'_{P_i} \neq s_{P_i}$ of P_i and for all strategy profiles s'_{-i} of players other than P_i , $u_i(s_{P_i} \cup s'_{-i}) + n(\lambda) > u_i(s'_{P_i} \cup s'_{-i})$ where $n(\cdot)$ is a negligible function and λ is the security parameter.*

ℓ -Repellence [9]. [9] provides transferable utility based threshold security definitions that are useful for disincentivizing coalitions up to certain sizes. We provide below the computational version of ℓ -repellence.

Definition 4 (Computational ℓ -Repellent Mechanism). *A joint strategy s_N is a computational ℓ -repellent equilibrium, if $\forall G \subseteq N$ with $|G| \leq \ell$ and for all strategies s'_G played by G , there exist a negligible function $n(\cdot)$ s.t. $u_G(s_N) + n(\lambda) > U_G(s'_G \cup s_{-G})$ where $s_N = s_G \cup s_{-G}$. For some functionality F , (Π, s_N) is a computationally ℓ -repellent mechanism for F , if s_N is a computational ℓ -repellent equilibrium of Π and the outcome of (Π, s_N) satisfies F .*

Merkle Hash Tree. A Merkle hash tree is a binary tree generated by hashing a set of data elements $E = (e_1, \dots, e_s)$. Each leaf with index i of the tree is constructed as $H(e_i)$ where $H : \{0, 1\}^* \rightarrow R$ is a collision resistant hash function, and R is a set whose each element can be encoded as a bit string and whose number of elements is $O(\text{poly}(\lambda))$. Each other node of the tree is constructed by $H(i, j)$ where i and j are the children of that node. The root hash is denoted by $z = MHRoot(E)$. We denote the Merkle proof of element e_i as $p_i = MHproof(E, e_i)$. This proof is composed of the hash values of the sibling nodes along the path between $H(e_i)$ and the root, and is used for proving the inclusion of an element in z with one minus negligible probability. $VerifyMHProof(z, i, e_i, p_i)$ is the verification algorithm that runs on input the root hash z , an index i , and the indexed element e_i and its proof p_i . This algorithm can be computed in complexity $O(\log s)$ to obtain an output *Success* or *Failure* depending on the success of the verification.

Computation trace. The computation is represented by a Turing machine $TM()$ with an input tape initially storing the input string. The reduced configuration rc_j of the computation state at the step j is assigned as $(s_j, h_j, v_j, MHRoot(t_j))$, where s_j , h_j , v_j , and t_j are the state, the head position, the tape at the given head, and the tape of the related TM configuration

[20]. We let the set $RC = (rc_1, \dots, rc_s)$ and the hash root of the computation $z = MHRoot(RC)$ as in [55].

MismatchCheck protocol. Proposed for use in two-party incentivized outsourced computation [55], this protocol is run among a checker (the smart contract in our setting) and two contractors C_i and C_j who are assigned to execute the same computation TM on some input x . The aim is to catch the cheater (set to lose) if only one of C_i and C_j is diligent (set to win) and the other one is lazy. This protocol always makes at least one contractor lose if their inputs are different (i.e., mismatching). Before the protocol is run, the checker receives the responses (y_i, z_i) and (y_j, z_j) from the contractors. The execution flow of *MismatchCheck* is given in Protocol ?? in Appendix ?. The protocol has $O(\log s)$ complexity where s is the number of computation steps. In our proposal, we use multiple executions of this protocol to obtain the final result in case of the absence of consensus in the submitted responses by the contractors. This protocol does not catch deviant strategies, where the player runs the diligent algorithm and then maliciously alters some of the reduced steps. This is because it is assumed that once the player goes with incorrect step all the steps afterwards will be spoiled as playing lazy.

Non-Interactive and Reusable-CRS Commitment Schemes: Commitment schemes (*Commit*, *Open*) are useful cryptographic primitives that enable Alice to commit to a value for Bob via *Commit* operation, and later open it via *Open* operation. The two traditional requirements for commitment schemes are being hiding (i.e., *Commit* does not reveal the committed value without *Open*) and binding (i.e., once *Commit* takes place, Alice cannot execute *Open* for different values). A commitment scheme is non-interactive if both *Commit* and *Open* costs one round, i.e., for each of them, one string is sent to Bob. The way that we utilize the commitment scheme in Section 3 requires commitments from multiple parties, and possibly many similar protocols may run concurrently. We stress that commitment schemes in the standard model are not secure by default in this setting as other users' commitments and their openings can be utilized for copying or generating different commitments. Instead, we expect the commitment scheme to satisfy the ideal functionality given in Protocol 1.1, i.e., the string commitment version of the multiple bit commitment ideal functionality given in [19]. Unlike the ones in standard model, $\mathcal{F}_{\text{MCOM}}$ ensures each commitment is generated as a completely blinded envelop and is bound to its generator.

UC-secure commitments are shown to be non-existent without a common reference string CRS (see ideal functionality in Protocol 1.2) by [19]. Therefore the ideal functionality given in Protocol 1.1 is the one with reusable-CRS for efficiency. Any non-interactive commitment scheme realizing $\mathcal{F}_{\text{MCOM}}$, e.g. [19, 23, 33, 35, 36], can be used for realizing our protocol. Concurrent commitment schemes are also shown to UC-secure by [58], therefore, the non-interactive ones [52, 57, 11] are also eligible. However, stand-alone non-malleable commitments [26, 25] should be shown to realize $\mathcal{F}_{\text{MCOM}}$ before their use.

Functionality $\mathcal{F}_{\text{MCOM}}$

Running with parties $P_1, \dots, P_n$ and an adversary $\mathcal{S}$, $\mathcal{F}_{\text{MCOM}}$ proceeds as follows.

1. On receiving a message $(\text{Commit}, \text{sid}, \text{cid}, P_i, P_j, s)$ from P_i , record the tuple $(\text{cid}, P_i, P_j, s)$ and send the message $(\text{Receipt}, \text{sid}, \text{cid}, P_i, P_j)$ to P_j and $\mathcal{S}$, where sid is the session id, cid is the commitment id, and $s \in \{0, 1\}^*$ is the committed string. Ignore subsequent $(\text{Commit}, \text{sid}, \text{cid}, \cdot)$ messages.
2. On receiving a message $(\text{Open}, \text{sid}, \text{cid}, P_i, P_j)$ from P_i , proceed as follows: If there exists a record for the tuple $(\text{cid}, P_i, P_j, s)$, then send the message $(\text{Open}, \text{sid}, \text{cid}, P_i, P_j, s)$ to P_j and $\mathcal{S}$. Otherwise, do nothing.

Protocol (1.1) The ideal functionality for multiple UC commitments

Functionality $\mathcal{F}_{\text{CRS}}$

Parameterized by a distribution D , $\mathcal{F}_{\text{CRS}}$ proceeds as follows.

1. When activated for the first time on input $(\text{value}, \text{sid})$, choose a value $d \leftarrow_R D$ and send d back to the activating party. In each other activation, return d to the activating party.

Protocol (1.2) The ideal functionality for the common reference string

3 Our Response Submission Protocol

The ideal functionality $\mathcal{F}_{\text{RS}}$. Protocol 1.3 provides the ideal functionality $\mathcal{F}_{\text{RS}}$ for the response submissions of the contractors. **Response** messages from the contractors to $\mathcal{F}_{\text{RS}}$ need to be sent before τ_{SC} . A contractor may send **Leak** messages to $\mathcal{F}_{\text{RS}}$, to model the action of leaking the response to another contractor. Once the response is submitted, it cannot be altered or re-submitted.

Protocol π_{RS} . Protocol 1.4 provides our proposed protocol π_{RS} to realize $\mathcal{F}_{\text{RS}}$ in the $\mathcal{F}_{\text{MCOM}}$ -hybrid model. π_{RS} runs with a single copy of $\mathcal{F}_{\text{MCOM}}$. Until the current time $\tau_C = \tau_F$, SC just records the obtained messages and then gets activated to run the algorithm for the output. We highlight that in the implementation of this protocol the value CRS will be required for the realization of the functionality $\mathcal{F}_{\text{MCOM}}$, which needs to be submitted to SC by the boss before the start of the protocol execution as part of the auxiliary information aux . We assume that commitments are tied to the unique identities of the participants and therefore cannot be copied without being noticed. Proof is in the Appendix.

Functionality $\mathcal{F}_{\text{RS}}$

Running with contractors $P_1, \dots, P_n$, the smart contract SC, and an adversary $\mathcal{S}$, $\mathcal{F}_{\text{RS}}$ proceeds as follows.

1. When the current time $\tau_C < \tau_{\text{SC}}$, on receiving a message $(\text{Response}, \text{sid}, P_i, (y, z))$ from P_i for the first time, record the tuple $(P_i, \text{sid}, (y, z))$. Send the message $(\text{Receipt}, \text{sid}, P_i)$ to $\mathcal{S}$. Ignore other $(\text{Response}, \text{sid}, P_i, \cdot)$ tuples. On receiving a message $(\text{Leak}, \text{sid}, P_i)$ from P_i , if there exists a record $(P_i, \text{sid}, (y, z))$, send $(\text{Leak}, \text{sid}, P_i, (y, z))$ to $\mathcal{S}$. Otherwise do nothing.
2. When $\tau_{\text{SC}} = \tau_C$, send each recorded $(P_i, \text{sid}, (y, z))$ tuple to $\mathcal{S}$.
3. When $\tau_C = \tau_F$, send each recorded $(P_i, \text{sid}, (y, z))$ tuple to SC.

Protocol (1.3) The ideal functionality for response submission of contractors

The Response Submission Protocol π_{RS}

Running with contractors $C_1, \dots, C_n$, the smart contract SC, and an adversary $\mathcal{H}$, π_{RS} proceeds as follows in the presence of one copy of $\mathcal{F}_{MCOM}$ functionality with session id sid_{MCOM} running with parties $P_1, \dots, P_n$, and SC.

1. When the current time $\tau_C < \tau_{SC}$, an honest P_i and an honest SC proceed as follows. Upon the first input (**Response**, $sid, P_i, (y, z)$), P_i picks a unique cid and sends the message (**Commit**, $sid_{MCOM}, cid, P_i, SC, (y, z)$) to $\mathcal{F}_{MCOM}$. Then, $\mathcal{F}_{MCOM}$ sends (**Commit**, sid_{MCOM}, cid, P_i, SC) to SC. P_i ignores other (**Response**, $*$) inputs. Upon the input (**Leak**, sid, P_i), P_i sends the message (**Open**, sid_{MCOM}, cid, P_i, SC) to $\mathcal{F}_{MCOM}$, which in turn sends (**Open**, $sid_{MCOM}, cid, P_i, SC, (y, z)$) to SC. SC just keeps appending all the messages coming from $\mathcal{F}_{MCOM}$ to its database.
2. When $\tau_{SC} \leq \tau_C < \tau_F$, an honest P_i and SC proceed as follows. P_i sends the message (**Open**, sid_{MCOM}, cid, P_i, SC) to $\mathcal{F}_{MCOM}$. If there is a related record, $\mathcal{F}_{MCOM}$ sends (**Open**, $sid_{MCOM}, cid, P_i, SC, (y, z)$) to SC. Again, SC just passively keeps appending all the messages coming from $\mathcal{F}_{MCOM}$ to its database.
3. When $\tau_C = \tau_F$, on its database, SC deduces each P_i 's first (**Commit**, sid_{MCOM}, cid, P_i, SC) message among the ones sent before τ_{SC} on its database. These messages are combined to obtain a table T with up to n entries. Then, SC generates another table T' with entries $(P_i, sid, (y, z))$ obtained from the corresponding (**Open**, $sid_{MCOM}, cid, P_i, SC, (y, z)$) entries sent in the interval (τ_{SC}, τ_F) for each (P_i, sid_{MCOM}, cid) on T . For the ones that are found, SC outputs each entry of T' .

Protocol (1.4) The $\mathcal{F}_{MCOM}$ -hybrid protocol for response submission

4 Our Incentivized Outsourced Computation Proposal

System Model: Essentially, the setting that we are interested in is a multi-party version of the two-party incentivized outsourced computation setting [7, 55] and the inclusion of smart contracts and rational coalitions on top of [54], as we describe in what follows.

There exist three types of entities: (i) a *boss*, who is interested in outsourcing the computation of a deterministic polynomial time⁴ Turing machine $TM()$ on an input tape x , (ii) a set N of n *contractors* that execute the computation to obtain the reward, and (iii) a *Smart Contract (SC)* that rules the system interactions between the parties on behalf of the boss, and handles the transactions. We can consider the boss as an honest designer of the mechanism rather than a player involved in it, as in [7, 54, 55]. She outsources the computation to the contractors and generates SC at the beginning of the game. Her goal is to obtain the correct computation result from the contractors at the end of the protocol, and she is willing to pay a total reward R for this goal.

We model a contractor as a rational party whose main interest is to maximize its utility. We bound each contractor to execute only a probabilistic polynomial time (PPT) interactive Turing machine (ITM).

Non-Collaborating and Collaborating Coalitions. The contractors may establish coalitions; thus the boss is naturally advised to pick the contractors

⁴ If it is a randomized algorithm, it should be converted to a deterministic one by providing the randomization script (e.g., pseudorandom seed) as part of the input.

in a manner that lowers the probability of coalitions. For simplicity, as usual [7, 54, 55], we assume the contractors are picked randomly from a pool. We identify two types of possible coalitions among the contractors: (i) the *non-collaborative setting*, where the members of a coalition together decide the strategy that each of them individually plays, among the available strategies, and (ii) the *collaborative setting*, where the members of a coalition can go beyond the individual strategies. In particular, they can simply decide that only one of them performs the computation (honestly or not), and then all of them submit the same result. We note that the previous work [9] only analyzes the non-collaborative setting. Following the transferable utility assumption, we assume a single total utility for the coalition. This allows our setting to address Sybil contractors in the sense that all Sybils can form a coalition and our theorems address this issue directly (except when all contractors picked for a particular job turn out to be Sybils).

Threat Model. The network is asynchronous, i.e., under adversarial control. Further, the communication can be eavesdropped by the adversary but cannot be modified (i.e., they are authenticated). Each party has a unique identity. The adversary can actively and adaptively corrupt any number of parties. SC is an ITM that is trusted for running the program that it has been given, but its computation and storage are public, so it cannot protect any private values. The communication with SC is over authenticated public channels. **Goals:**

1. With high probability, the boss receives the correct computation result (considered by [7, 54, 55, 9]).
2. The contractors are not discouraged by setting a high fine to reward ratio (considered by [7]).
3. There is an incentive for contractors to act honestly, i.e., an honestly acting contractor gets rewarded (considered by [7, 54, 55, 9]).
4. There is an incentive for contractors to submit the response early (not considered before).
5. Contractors playing the honest strategy are incentivized to help catch deviating contractors.
6. The mechanism is safe against collaborating contractors, i.e., the workload of computation can be shared within the coalition (only non-collaborating coalitions are considered by [9]).
7. The workload to SC in case of mismatching submissions is kept low for the budget of the boss (considered by [55]).

4.1 Our Delegate Protocol

To prepare for outsourcing, the boss specifies SC for the involved contractors, the computation $TM()$ to be executed, and the input x . SC collects deposits from the boss and the contractors. In accordance with the protocol π_{RS} (see Protocol 1.4), until a predetermined time τ_{SC} , SC passively collects computation results from the contractors. The computation results are submitted as **Commit** to (y_i, z_i) by each contractor P_i , where y_i is the computation result and z_i is the Merkle hash root of all computation steps (see Section 2). Then, until another predetermined time τ_F , the contractors send **Open** of their submitted responses to SC. After

τ_F , SC checks all the submissions, determines the result of the computation, and delivers fines and rewards. It also returns the boss’s residual deposit if there is any remaining. Finally, it sends the computation result to the boss. Protocol 1.5 shows the high-level flow of **Delegate**.

Notation (extended from [55]) TM is the function to be computed, picked by the boss. x is its input. aux is the auxiliary information submitted to SC at the beginning of the protocol by the boss. y is the computation result and z is the Merkle root of the computation steps, submitted by a contractor. r is reward, f is fine, τ_{SC} is the deadline for committing to the response (y, z) . τ_F is the deadline for opening the committed response. τ_C is the current time.

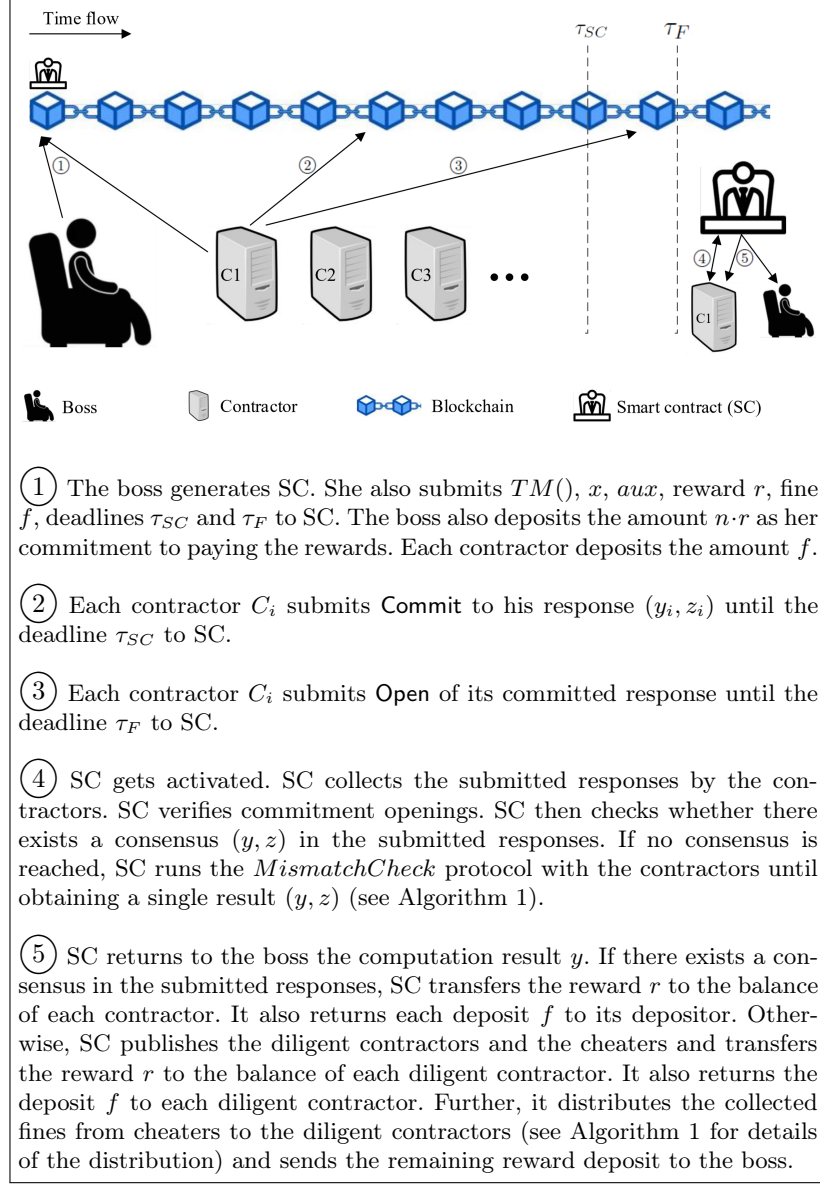
Algorithm 1 shows the algorithm of SC throughout the protocol. The algorithm activates at τ_F and collects fine deposits and responses submitted by the contractors. If no consensus in the submitted responses exists, SC executes the for loop in the mismatch resolution part. At each round of the for loop, the algorithm sets two of the submitted responses to compete by *MismatchCheck* executions (see Section 2). In each of these executions, submitters of the two responses are involved. When a submitter of a response loses, the algorithm continues by the next submitter of that response, until all submitters of a response are set to lose. Then that response gets eliminated. This way we make sure a submitted response can only get eliminated after all of its submitters are involved in *MismatchCheck* executions as a submitter of a response can be part of a hidden separate coalition and can lose *MismatchCheck* on purpose. After one response gets eliminated the for loop continues to the next round by picking another response to compete with the winner of the previous round.

Unlike the previous works [54, 55, 9], the fine deposits by cheaters will be delivered to the diligent contractors that get actively involved in catching the cheaters in *MismatchCheck* executions. The number of executions that a diligent contractor gets involved in is less than or equal to the number of fine gains that he will receive. We consider this as a further incentivization to get involved and to stay online, as long as f is greater than the *MismatchCheck* execution cost for the diligent contractor. Further, the algorithm takes into account the response submission order in assigning *MismatchCheck* executions, as an incentivization of early submission by the diligent contractors. If a diligent contractor submits earliest, stays online, and joins each *MismatchCheck* asked by SC, he gets all the fines of the cheaters. An honest contractor algorithm is provided as Algorithm 2.

5 Game Theoretical Analysis of Delegate

Strategies of Contractors Outside Coalition: We define the strategy set in the setting without any coalition based on the strategies given in [7, 54, 55]. The available actions for each contractor are briefly as follows.

- Regarding computation of the task:
 1. *Diligent computation*. This is the preferred action by the boss. The contractor runs the correct computation honestly and submits the result in the given time. The cost of the computation is denoted as $\text{cost}(d)$.



Protocol (1.5) Our Delegate protocol flow

Algorithm 1: Complete algorithm of SC

Activate at τ_F **Initial Check:** Check the sent blinded responses and their openings and prepare a table for each contractor with entries $(P_i, (y_i, z_i))$ where (y_i, z_i) is the first submitted commitment by P_i .**for each contractor P_i do** **if** P_i did not submit a commitment or its opening **then** | $Cheaters.add(P_i)$ **else** | $Response(y_i, z_i).add(P_i)$ **if $\forall i, j (y_i, z_i) = (y_j, z_j)$ then** | $ResultingResponse \leftarrow Response(y, z)$ | Go to **Delivery****Mismatch resolution:**Initialize $ResultingResponse$ as the most common $Response(y, z)$ $ResultingResponse[0].count \leftarrow Cheaters.size$ **for each $Response(y, z)$ do** $i \leftarrow 0, j \leftarrow 0$ Execute $MismatchCheck$ with $ResultingResponse[i]$ and $Response(y, z)[j]$, during execution: **if** $ResultingResponse[i]$ fails to comply **or** loses $MismatchCheck$
 then **if** $i < ResultingResponse.size$ **then** | $i \leftarrow i + 1$, restart $MismatchCheck$ **else** | $Cheaters.add(ResultingResponse)$ | $Response(y, z)[j].count \leftarrow Cheaters.size$ | $ResultingResponse \leftarrow Response(y, z)$ **else if** $Response(y, z)(j)$ fails to comply **or** loses $MismatchCheck$
 then **if** $j < Response(y, z).size$ **then** | $j \leftarrow j + 1$, restart $MismatchCheck$ **else** | $ResultingResponse[j].count += Response(y, z).size$, | $Cheaters.add(Response(y, z))$ **Delivery:** Publish $ResultingResponse$, $Cheaters$, $Quitters$ Deliver $(n - ResultingResponse.size) \cdot r$ to the bossDeliver $r + f + j.count \cdot f$ to each $j \in ResultingResponse$

Algorithm 2: Algorithm of Honest Contractor

Activate at the protocol start: Deposit f to SC and learn $TM()$ and x . Compute $TM(x)$ diligently to obtain (y, z) . Submit **Commit** to the response (y, z) .**Activate at τ_{SC} :** Submit **Open** of the commitment.**Activate at τ_F :** Comply with each $MismatchCheck$ request from SC.

2. *Lazy algorithm computation.* This action corresponds to choosing a lazy algorithm from the set $L = (l_1, \dots, l_m)$ where $m \in O(\text{poly}(\lambda))$. In a lazy strategy with the algorithm l_i , the contractor runs l_i whose cost is defined as $\text{cost}(l_i) < \text{cost}(d)$. In presence of a single diligent contractor in the system, the probability of the lazy algorithm l_i producing a successful result is defined as $q_i < \mathfrak{n}(\lambda)$ for some negligible function $\mathfrak{n}(\cdot)$, thanks to the *MismatchCheck* protocols executed by SC. We highlight that this is a generalization of the previous “lazy q -algorithm” of [54, 55, 9] and “Guess” strategy by [55]. We also highlight that, unlike the previous works, we do not limit the contractors to play a single lazy algorithm. Note that the algorithms with $\text{cost}(l_i) \geq \text{cost}(d)$ are simply dominated by the diligent algorithm and excluded from the consideration.
- Regarding response submission (based on actions in $\mathcal{F}_{RS}$):
 1. *Honest submission.* This corresponds to submitting the output of the computation algorithm (can be a lazy one).
 2. *Leaking.* This is a deviant submission action. The submitter of a response deliberately leaks the submitted response. We disincentivize leaking.
 3. *Copy strategies.* The copy attack [5] is that the contractor waits for some response from the other contractors on the SC, then copies and submits that response, as if it is the contractor’s own result. [55] describes combined strategies including copy attack and diligent and lazy strategies. In this work, we proposed a UC-secure response submission protocol that allows any PPT contractor to conduct this attack with at most a negligible probability, as long as the original submitter does not leak it.
 - Regarding *Mismatch* protocol executions:
 1. *Not complying.* This includes trying to cheat during a *Mismatch* execution or failing to send messages in time.
 2. *Complying.* Honestly preparing and sending the messages in time per the protocol requirements.

Analysis of Leaking and *MismatchCheck* Actions: Table 3 shows the utility matrix of a diligent contractor based on the leaking decision. Not leaking is computationally dominant, as it can be better if there exist non-diligent contractors. In such a case, the contractor can obtain an extra reward by cheater hunting. In this case, leaking results in utility $r - \text{cost}(d)$ due to copiers.

Others $\setminus$ This	Do not leak	Leak
All Diligent and Honest Submission	$r - \text{cost}(d)$	$r - \text{cost}(d)$
Some Lazies and Copiers	$\geq r - \text{cost}(d)$	$r - \text{cost}(d)$

Table (3) Utility matrix of a diligent contractor based on leaking.

Analysis Without Coalitions: Now we allow each contractor to choose only Diligent and Lazy as strategies (i.e., honest submission and complying is ensured), and disallow any group of contractors from forming coalitions. Therefore, this setting is also interesting if there is no SC and the boss is managing the verification, as in [7, 54, 9]. We start by showing that by adjusting the parameters, one can ensure all-diligent is the unique computational Nash equilibrium in this setting. Table 4 provides the utility matrix of the contractor. All proofs

are in the Appendix. In all the theorems below, we obviously assume that the rewards, fines, and extra rewards are all noticeable (not negligible).

Others $\setminus$ This	Diligent	Lazy l_i
All Diligent	$r - \text{cost}(d)$	$r \cdot q_i - f(1 - q_i) - \text{cost}(l_i)$
k Diligent	$\geq r - \text{cost}(d)$	$\leq r \cdot q_i - f(1 - q_i) - \text{cost}(l_i)$
All Lazy L'	$r + f \sum_{l_j \in L'} (1 - q_j) - \text{cost}(d)$	*depends on l_i and L'

Table (4) Utility matrix of a contractor if no coalition exists, only honest submission and complying is allowed. k should satisfy $0 < k < n - 1$.

Theorem 1. *If the hash function utilized in Merkle hash tree is collision resistant and the boss sets the fine and reward as $f > \frac{r}{(n-1)(1-n(\lambda))}$ for all negligible functions $n(\cdot)$ where λ is a security parameter, then in the setting described in this subsection, the strategy with diligent computation, honest submission, and complying during MismatchCheck for each player is the unique computational Nash equilibrium.*

We show that all diligent computation is still a computational Nash equilibrium and that there does not exist any Nash equilibrium where no contractor is honest. Note that a simple setting would be $f = r/(n - 1)$ satisfying the requirement. We allow ourselves by not requiring all diligent computation and honest submission without leaking strategy as the unique computational Nash equilibrium anymore. This is forced as leaking does not decrease the utility of any diligent player. Thus, the unique equilibrium above does not take into account leaking behavior. Hence, it is OK to leak as long as the result is honest. Yet the boss obtains the correct output (guaranteed by the Theorems 2 and 3).

Theorem 2. *If the hash function utilized in Merkle hash is collision resistant, and the response submission protocol π_{RS} given in Protocol 1.4 securely emulates the ideal functionality $\mathcal{F}_{RS}$ given in Protocol 1.3; then in the setting described in this subsection, each player following diligent computation, honest submission, and complying without leaking is a computational Nash equilibrium.*

Theorem 3. *If the hash function utilized in Merkle hash is collision resistant, and the response submission protocol π_{RS} given in Protocol 1.4 securely emulates the ideal functionality $\mathcal{F}_{RS}$ given in Protocol 1.3, and the boss sets the fine and reward as $f > \frac{r}{(n-1)(1-n(\lambda))}$ for all negligible functions $n(\cdot)$ where λ is a security parameter; then in the setting described in this subsection, there exists no computational Nash equilibrium where at least one player does not follow the diligent computation and honest submission with or without leaking strategies.*

Analysis With Non-Collaborating Coalitions: By using the computational ℓ -repellence definition, we show that our mechanism is secure against coalitions with up to $n - 1$ contractors.

Definition 5 (non-collaborating diligent strategy). *Non-collaborating diligent strategy is defined as a non-collaborating coalition strategy where the diligent algorithm is executed by each player in the coalition, the output is submitted honestly, and during MismatchCheck the first submitter of the response complies.*

Theorem 4. *If the hash function utilized in Merkle hash is collision resistant, and the response submission protocol π_{RS} securely emulates the ideal functionality $\mathcal{F}_{RS}$, then in the setting provided in this subsection, all non-collaborating diligent strategy is computationally $(n - 1)$ -repellent.*

Analysis With Collaborating Coalitions: A coalition may benefit from reducing computation costs by parallelizing the task (or letting only one player handle it), resulting in a total cost of $\text{cost}(d)$ for a coalition of size k , instead of $k \cdot \text{cost}(d)$, to play the all diligent strategy. Assuming the boss is only interested in obtaining the correct result (not necessarily in every player executing the computation), we obtain an equilibrium.

Definition 6 (collaborating diligent strategy). *Collaborating diligent strategy is defined as a collaborating coalition strategy where the diligent algorithm is executed once in the coalition, the output is submitted honestly by all players in the coalition without leaking, and during `MismatchCheck` the first submitter of the response complies.*

We acknowledge that this is a slight abuse of the previous computational ℓ -repellence definition, where the desired coalition strategy was expected to be the collection of the desired strategies of each contractor inside it. Yet, it is justified due to the following reasons: (1) cost cancellations due to interactions of coalitional partners are inevitable as the same computation is outsourced, (2) the boss only cares about the outcome, i.e., obtaining the correct result.

Theorem 5. *If the hash function utilized in Merkle hash is collision resistant, and the response submission protocol π_{RS} securely emulates the ideal functionality $\mathcal{F}_{RS}$, then in the setting provided in this subsection, all collaborating diligent strategy is computationally $(n - 1)$ -repellent.*

6 Implementation and Results

Implementation and Test Environment. We have a proof-of-concept implementation of our protocol using Python and web3.py. Also, we implement the algorithms for SC with Solidity on Remix IDE. For simplicity, we model the given function f as a random access computer as in [22] instead of Turing machines. We highlight that the contractors still execute the computation step-by-step and the Merkle hash tree is computed from these steps. Regarding the function f , we pick finding the x -th Fibonacci number by brute force, where x is given as input to f . The implementation of `MismatchCheck` use SHA-3 (256 bits) for constructing the Merkle hash tree. Also, the UC commitment scheme used for π_{RS} is [19], which uses SHA-2 and SHA-3 (256 bits) for clawfree trapdoor permutations (for simplicity) and the Cramer-Shoup encryption scheme [17] as CCA-secure public key encryption. The tests for the contractor implementation are executed on Apple M1 CPU 16 GB RAM. The tests for SC implementation are conducted on Truffle Ganache environment. For tests of the `MismatchCheck`

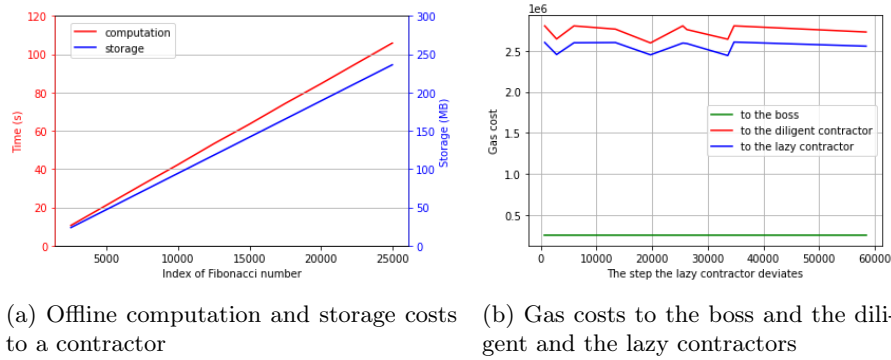


Fig. (1) The costs of our protocol (excluding the response submission protocol). The costs in 1a are w.r.t. index of the Fibonacci number to be computed. The costs in 1b are w.r.t the deviating step of the lazy contractor.

protocol, we take two contractors one diligent and the other one lazy deviating at a step picked uniformly at random. Our tests are repeated 5 times averaged.

Our results include the cost of computation of a function f (we exemplify as $f(x) := [x^{th} \text{ Fibonacci number found by brute force}]$) together with the construction of the Merkle tree hash z and executing *MismatchCheck* protocol. These costs include the offline computation cost of a contractor, and, *only in the case of existence of a lazy contractor*, the online gas costs to a diligent contractor, a lazy contractor, and the SC (i.e., the boss). We stress that online gas costs will be only incurred if there exists a lazy contractor, who will be the payer of those costs as a fine (but if all participants are rational, they will all be diligent anyways). Figure 1 shows our results. Our costs do not include the costs of the UC response submission protocol, which is highly dependent on the commitment scheme used.

Limitations: We acknowledge that our performance evaluation is simplistic, and a full implementation including UC commitments with a more realistic workload would strengthen the contribution. Such further experiments should also highlight the impact of irrational contractors in the system, which is currently outside our system model. Secondly, while our solution addresses Sybils since they can be thought as a form of a coalition, it does not take into account the case where all contractors picked for a particular job consist of Sybils. Finally, [71] released a tool for automatically analyzing ℓ -repellence. We leave its usage as potential future work.

References

1. Abraham, I., Dolev, D., Gonen, R., Halpern, J.: Distributed computing meets game theory: Robust mechanisms for rational secret sharing and multiparty computation. In: ACM PODC (2006)

2. Abraham, I., Dolev, D., Halpern, J.Y.: Distributed protocols for leader election: A game-theoretic perspective. In: DISC (2013)
3. Afanasev, M.Y., Fedosov, Y.V., Krylova, A.A., Shorokhov, S.A.: An application of blockchain and smart contracts for machine-to-machine communications in cyber-physical production systems. In: IEEE ICPS (2018)
4. Aiyer, A.S., Alvisi, L., Clement, A., Dahlin, M., Martin, J.P., Porth, C.: Bar fault tolerance for cooperative services. *ACM SIGOPS Oper. Syst. Rev.* **39**(5), 45–58 (2005)
5. Avizheh, S., Nabi, M., Safavi-Naini, R., Venkateswarlu K., M.: Verifiable computation using smart contracts. In: ACM CCSW (2019)
6. Avizheh, S., Nabi, M., Safavi-Naini, R.: Refereed delegation of computation using smart contracts. *IEEE TDSC* (2024)
7. Belenkiy, M., Chase, M., Erway, C.C., Jannotti, J., Küpçü, A., Lysyanskaya, A.: Incentivizing outsourced computation. In: NetEcon (2008)
8. Bellare, M., Goldreich, O.: On defining proofs of knowledge. In: CRYPTO (1993)
9. Biçer, O., Yıldız, B., Küpçü, A.: m-stability: Threshold security meets transferable utility. In: ACM CCSW (2021)
10. Bingöl, M.A., Biçer, O., Kiraz, M.S., Levi, A.: An Efficient 2-Party Private Function Evaluation Protocol Based on Half Gates. *The Computer Journal* **62**(4) (2018)
11. Bitansky, N., Lin, H.: One-message zero knowledge and non-malleable commitments. In: TCC (2018)
12. Biçer, O., Bingöl, M.A., Kiraz, M.S., Levi, A.: Highly efficient and re-executable private function evaluation with linear complexity. *IEEE TDSC* (2020)
13. Bracha, G., Toueg, S.: Asynchronous consensus and broadcast protocols. *J. ACM* **32**(4), 824–840 (Oct 1985)
14. Braun, B., Feldman, A.J., Ren, Z., Setty, S., Blumberg, A.J., Walfish, M.: Verifying computations with state. In: ACM SOSP (2013)
15. Brenguier, R.: Robust equilibria in mean-payoff games. In: FoSSaCS (2016)
16. Buterin, V.: Ethereum white paper: A next-generation smart contract and decentralized application platform (2013)
17. Camenisch, J., Stadler, M.: Efficient group signature schemes for large groups. In: CRYPTO (1997)
18. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: IEEE FOCS (2001)
19. Canetti, R., Fischlin, M.: Universally composable commitments. In: CRYPTO (2001)
20. Canetti, R., Riva, B., Rothblum, G.N.: Practical delegation of computation using multiple servers. In: ACM CCS (2011)
21. Cheng, R., Zhang, F., Kos, J., He, W., Hynes, N., Johnson, N., Juels, A., Miller, A., Song, D.: Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In: EuroS&P (2019)
22. Cook, S.A., Reckhow, R.A.: Time bounded random access machines. *Journal of Computer and System Sciences* **7**(4), 354–375 (1973)
23. Damgård, I., Groth, J.: Non-interactive and reusable non-malleable commitment schemes. In: ACM STOC (2003)
24. Dani, V., Movahedi, M., Rodriguez, Y., Saia, J.: Scalable rational secret sharing. In: ACM PODC (2011)
25. Di Crescenzo, G., Ishai, Y., Ostrovsky, R.: Non-interactive and non-malleable commitment. In: ACM STOC (1998)
26. Dolev, D., Dwork, C., Naor, M.: Non-malleable cryptography. In: ACM STOC (1991)

27. Dong, B., Wang, H., Monreale, A., Pedreschi, D., Giannotti, F., Guo, W.: Authenticated outlier mining for outsourced databases. *IEEE TDSC* **17**(2), 222–235 (2020)
28. Dong, C., Wang, Y., Aldweesh, A., McCorry, P., van Moorsel, A.: Betrayal, distrust, and rationality: Smart counter-collusion contracts for verifiable cloud computing. In: *ACM CCS* (2017)
29. Douceur, J.R.: The sybil attack. In: *IPTPS* (2002)
30. Du, W., Murugesan, M., Jia, J.: *Uncheatable Grid Computing*. Chapman & Hall/CRC (2010)
31. Farrell, J., Rabin, M.: Cheap talk. *Journal of Economic Perspectives* **10**(3), 103–118 (September 1996)
32. Feldman, M., Papadimitriou, C., Chuang, J., Stoica, I.: Free-riding and whitewashing in peer-to-peer systems. In: *ACM PINS* (2004)
33. Fischlin, M., Libert, B., Manulis, M.: Non-interactive and re-usable universally composable string commitments with adaptive security. In: *ASIACRYPT* (2011)
34. Fuchsbaauer, G., Katz, J., Naccache, D.: Efficient rational secret sharing in standard communication networks. In: *TCC* (2010)
35. Fujisaki, E.: All-but-many encryption: A new framework for fully-equipped uc commitments. In: *ASIACRYPT* (2014)
36. Fujisaki, E.: Improving practical uc-secure commitments based on the ddh assumption. In: *SCN* (2016)
37. Garay, J., Katz, J., Maurer, U., Tackmann, B., Zikas, V.: Rational protocol design: Cryptography against incentive-driven adversaries. In: *IEEE FOCS* (2013)
38. Gennaro, R., Gentry, C., Parno, B.: Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In: *CRYPTO* (2010)
39. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: *ACM STOC* (2009)
40. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game. In: *ACM STOC* (1987)
41. Goldreich, O., Micali, S., Wigderson, A.: Proofs that yield nothing but their validity or all languages in np have zero-knowledge proof systems. *J. ACM* **38**(3) (1991)
42. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating computation: Interactive proofs for muggles. *J. ACM* **62**(4) (2015)
43. Goldwasser, S., Micali, S., Rackoff, C.: The knowledge complexity of interactive proof-systems. In: *ACM STOC* (1985)
44. Golle, P., Mironov, I.: Uncheatable distributed computations. In: *CT-RSA* (2001)
45. Gordon, S.D., Katz, J.: Rational secret sharing, revisited. In: *SCN* (2006)
46. Halpern, J.Y., Vilaça, X.: Rational consensus: Extended abstract. In: *ACM PODC* (2016)
47. Heller, Y.: Ex-ante and ex-post strong correlated equilibrium. MPRA Paper 7717, University Library of Munich, Germany (Mar 2008)
48. Jakobsen, T.P., Nielsen, J.B., Orlandi, C.: A framework for outsourcing of secure computation. In: *CCSW* (2014)
49. Jiang, X., Tian, Y.: How to construct rational protocols with nash equilibrium consistency in the uc framework. *Cryptology ePrint Archive*, Report 2019/1398 (2019)
50. Kamara, S., Mohassel, P., Raykova, M.: Outsourcing multi-party computation. *Cryptology ePrint Archive*, Report 2011/272 (2011)
51. Kamara, S., Küpçü, A.: Dogfish: Decentralized optimistic game-theoretic file sharing. In: *ACNS* (2018)

52. Khurana, D., Sahai, A.: How to achieve non-malleability in one or two rounds. In: IEEE FOCS (2017)
53. Kothapalli, A., Miller, A., Borisov, N.: Smartcast: An incentive compatible consensus protocol using smart contracts. In: FC (2017)
54. Küpçü, A.: Incentivized outsourced computation resistant to malicious contractors. IEEE TDSC **14**(6) (2017)
55. Küpçü, A., Safavi-Naini, R.: Smart contracts for incentivized outsourcing of computation. In: ESORICS CBT (2021)
56. Li, H., Clement, A., Wong, E., Napper, J., Roy, I., Alvisi, L., Dahlin, M.: Bar gossip. In: OSDI (2006)
57. Lin, H., Pass, R., Soni, P.: Two-round and non-interactive concurrent non-malleable commitments from time-lock puzzles. In: IEEE FOCS (2017)
58. Lin, H., Pass, R., Venkitasubramaniam, M.: A unified framework for concurrent security: Universal composability from stand-alone non-malleability. In: ACM STOC (2009)
59. Loftus, J., Smart, N.P.: Secure outsourced computation. In: AFRICACRYPT (2011)
60. Lysyanskaya, A., Triandopoulos, N.: Rationality and adversarial behavior in multi-party computation. In: CRYPTO (2006)
61. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. In: IEEE SP (2013)
62. Parno, B., Raykova, M., Vaikuntanathan, V.: How to delegate and verify in public: Verifiable computation from attribute-based encryption. In: TCC (2012)
63. Peters, H.: Cooperative games with transferable utility. In: Game Theory: A Multi-Level Approach. Springer Berlin Heidelberg (2008)
64. Premnath, S.N., Haas, Z.J.: Security and privacy in the internet-of-things under time-and-budget-limited adversary model. IEEE Wireless Commun. Lett. **4**(3) (2015)
65. Sadeghi, A.R., Schneider, T., Winandy, M.: Token-based cloud computing. In: TRUST (2010)
66. Setty, S., McPherson, R., Blumberg, A.J., Walfish, M.: Making argument systems for outsourced computation practical (sometimes). In: NDSS (01 2012)
67. Setty, S., Vu, V., Panpalia, N., Braun, B., Blumberg, A.J., Walfish, M.: Taking proof-based verified computation a few steps closer to practicality. In: Usenix Sec. (2012)
68. Susilo, W., Guo, F., Zhao, Z., Jiang, Y., Ge, C.: Secure Replication-Based Outsourced Computation Using Smart Contracts . IEEE Transactions on Services Computing **16**(05), 3711–3722 (2023)
69. Szajda, D., Lawson, B., Owen, J.: Hardening functions for large scale distributed computations. In: IEEE SP (2003)
70. Tang, W., Liu, E., Ni, W., Qu, X., Huang, B., Li, K., Niyato, D., Jamalipour, A.: Game-theoretic incentive mechanism for blockchain-based federated learning. IEEE Transactions on Mobile Computing (2025)
71. Taweel, A., Yıldız, B., Küpçü, A.: Gamu blue: A practical tool for game theory security equilibria. In: IEEE/IFIP NOMS Mini Conference (2024)
72. Thaler, J., Roberts, M., Mitzenmacher, M., Pfister, H.: Verifiable computation with massively parallel interactive proofs. In: USENIX HotCloud (2012)
73. Walfish, M., Blumberg, A.J.: Verifying computations without reexecuting them. Commun. ACM **58**(2), 74–84 (Jan 2015)
74. Wang, Y., Wang, H., Xu, Q.: Rational secret sharing with semi-rational players. Int. J. Grid Util. Comput. **3**(1) (2012)

75. Xiao, Y., Zhang, N., Li, J., Lou, W., Hou, Y.T.: Privacyguard: Enforcing private data usage control with blockchain and attested off-chain contract execution. In: ESORICS (2020)

A Proofs of Theorems

Theorem 6. *The protocol π_{RS} given in Protocol 1.4 securely emulates the ideal functionality $\mathcal{F}_{RS}$ given in Protocol 1.3 in $\mathcal{F}_{MCOM}$ -hybrid model.*

Proof (of Theorem 6). We construct an ideal world PPT adversary $\mathcal{H}$ that interacts with the environment $\mathcal{Z}$, the dummy parties $P_0, \dots, P_n$, SC, and the ideal functionality $\mathcal{F}_{RS}$ in an ideal execution. $\mathcal{H}$ is divided into two internal parts, a copy of the real world PPT adversary $\mathcal{A}$ in a black-box fashion and a PPT simulator $\mathcal{S}$. Throughout the ideal execution, $\mathcal{S}$ simulates a real execution of π_{RS} for $\mathcal{A}$ by acting as an interface between $\mathcal{A}$ and the outside of $\mathcal{H}$. $\mathcal{S}$ is allowed to modify the messages between $\mathcal{A}$ and the outside world, with the exception that whenever $\mathcal{A}$ outputs a message to $\mathcal{Z}$, $\mathcal{S}$ just forwards this message to $\mathcal{Z}$.

1. Throughout the whole execution $\mathcal{S}$ runs an internal copy $\mathcal{F}'_{MCOM}$ of $\mathcal{F}_{MCOM}$ with the session id sid_{MCOM} .
2. When the current time $\tau_C < \tau_{SC}$:
 - (a) If at some point $\mathcal{S}$ receives $(Receipt, sid, P_i)$ from $\mathcal{F}_{RS}$, $\mathcal{S}$ picks a unique cid and gives the tuple $(Receipt, sid_{MCOM}, cid, P_i, SC)$ to $\mathcal{A}$. $\mathcal{S}$ also adds (P_i, cid) to a list L .
 - (b) If at some point $\mathcal{S}$ receives $(Leak, sid, P_i, (y, z))$ from $\mathcal{F}_{RS}$, $\mathcal{S}$ gives the tuple $(Open, sid_{MCOM}, cid, P_i, SC, (y, z))$ to $\mathcal{A}$.
 - (c) $\mathcal{S}$ simulates a corrupted party P_i as follows. For any $(Commit, sid_{MCOM}, cid, P_i, SC, (y, z))$ command, if $(P_i, \cdot) \notin L$, $\mathcal{S}$ sends $(Response, sid, P_i, (y, z))$ to $\mathcal{F}_{RS}$. For any $(Open, sid_{MCOM}, cid, P_i, SC)$ command, if $(P_i, cid) \in L$, $\mathcal{S}$ sends $(Leak, sid, P_i)$ to $\mathcal{F}_{RS}$. For all other cases, $\mathcal{S}$ utilize $\mathcal{F}'_{MCOM}$ to simulate $\mathcal{A}$'s view.
 - (d) $\mathcal{S}$ simulates internal transcripts of a corrupted SC via a database generated by appending all messages of the form $(Receipt, \cdot, \cdot, \cdot, SC)$ $(Open, \cdot, \cdot, \cdot, SC, \cdot)$ that $\mathcal{S}$ gives to $\mathcal{A}$. Internal transcripts of any other corrupted party P_i are simulated by using the input tape of the corresponding ideal dummy party (i.e., for the messages to $\mathcal{F}_{RS}$) and the cid values generated for that party by $\mathcal{S}$.
3. When $\tau_C = \tau_{SC}$, $\mathcal{S}$ receives each recorded $(P_i, sid, (y, z))$ tuple from $\mathcal{F}_{RS}$.
4. When $\tau_{SC} < \tau_C < \tau_F$:
 - (a) $\mathcal{S}$ simulates an honest party P_i 's $(Open, sid_{MCOM}, cid, P_i, SC)$ call, by giving to $\mathcal{A}$ the corresponding message, if it received a related record.
 - (b) For all other queries to $\mathcal{F}_{MCOM}$ by corrupted parties, $\mathcal{S}$ keeps simulating $\mathcal{A}$'s view via its internal copy of $\mathcal{F}_{MCOM}$.
 - (c) Again, internal transcripts are simulated as above.
5. When $\tau_C = \tau_{SC}$, $\mathcal{S}$ simulates a corrupted SC by running the algorithms given in the 3-rd Step of Protocol 1.4. Note that an honest SC does not make any call for any outside functionality at this step.

The view of $\mathcal{A}$ in the above simulation is computationally indistinguishable from the one in a real world protocol execution. Also, the view of $\mathcal{Z}$ in the ideal world interacting with $\mathcal{H}$ is computationally indistinguishable from the one in the real protocol execution interacting with $\mathcal{A}$.

Proof (of Theorem 1). There exist two main cases that we need check: (1) whether all diligent, honest submission, complying is a computational Nash equilibrium, (2) whether there exist any other computational Nash equilibrium than all diligent. Regarding the case (2), we use subcases to show no other computational Nash equilibria exist in the system.

Case (1): Check whether all diligent, honest submission, complying is a computational Nash equilibrium. If all the other contractors choose the diligent strategy, w.l.o.g, the utility of a contractor from the diligent strategy and that from a lazy strategy are $r - \text{cost}(d)$ and $r \cdot q_i - f(1 - q_i) - \text{cost}(l_i)$, respectively. Regarding the latter utility, we have $r \cdot q_i - f(1 - q_i) - \text{cost}(l_i) < -f - \text{cost}(l_i) + v_1$ for some negligible utility v_1 . Hence, if $r - \text{cost}(d) > -f - \text{cost}(l_i) + v_1$, we have the utility of diligent strategy to be greater for P . We calculate $r - \text{cost}(d) + \text{cost}(l_i) > r - r = 0 > -f + v_1$. Therefore, for all negligible utilities v , it is sufficient to have $f \geq v$ to achieve this as the computational Nash equilibrium.

Case (2): Check for existence of any other computational Nash equilibrium. If k of the contractors choose the diligent strategy where $0 < k < n$, the utility of a lazy P from the diligent strategy and that from any lazy strategy l_i are lower bounded by $r - \text{cost}(d)$ and upper bounded by $r \cdot q_i - f(1 - q_i) - \text{cost}(l_i) + v_2$ for some negligible utility v_2 , respectively. The former utility may increase compared to Case (1), because the contractor may gain some of the fines. Also, the latter utility may *only* increase compared to Case (1), if the lazy algorithm outputs a correct value and P gain some of the fines. As this occurs with negligible probability, the maximum increase in utility is bounded by a negligible v_2 . Regarding the latter utility, we have $r \cdot q_i - f(1 - q_i) - \text{cost}(l_i) + v_2 < -f - \text{cost}(l_i) + v_3$ for some negligible utility v_3 . The rest of the calculation is simply carried as in Case (1). Therefore, for all negligible utilities v , if $f > v$ diligent utility dominates the lazy one.

For the remaining, we need to show there is no computational Nash equilibrium in which no contractor plays the diligent strategy. Suppose there exists a computational Nash equilibrium in which no contractor plays the diligent strategy. If all the contractors are playing the same lazy strategy l_i , for each contractor the inequality $r + (n - 1)(1 - q_i)f - \text{cost}(d) \leq r - \text{cost}(l_i)$ is necessary, as otherwise this is not a computational Nash equilibrium. Then, if $(n - 1)(1 - q_i)f > \text{cost}(d) - \text{cost}(l_i)$, the computational Nash equilibrium is spoiled. It is straightforward to deduce that $f > \frac{r}{(n-1)(1-\mathfrak{n}(\lambda))}$ for all negligible functions $\mathfrak{n}(\cdot)$ is sufficient. This is because, the lowest $\text{cost}(l_i)$ would be 0, and the highest $\text{cost}(d)$ would be r since it does not make sense to perform the job if the cost is higher than the reward. If all the contractors are not playing the same lazy strategy, then one of these strategies (say l_i) would result in a utility lower bounded $r - \text{cost}(l_i)$, and each l_j of the others would result in a utility

$-f - \text{cost}(l_j) + v_4$ where v_4 is a negligible utility resulting from the multiplication of negligible probability that l_j outputs the same response with l_i and the resulting rewards and fine gains. Therefore, for all negligible utilities v , it is sufficient to have $f > v$, for allowing any contractor to convert l_i and receive a utility increase. This conflicts the supposed computational Nash equilibrium.

Proof (of Theorem 2). If all the other contractors follow diligent computation and honest submission without leaking, compared to honest submission by copy attack, the contractor can only increase its utility by a negligible utility. Further, in this case, by leaking his response the contractor does not change his utility.

Proof (of Theorem 3). Suppose there exists a computational Nash equilibrium in which no contractor plays the diligent strategy. Then some contractors play the same lazy strategy l_i , each obtaining utility $r - \text{cost}(l_i)$, whereas others may play copy thanks to some leaking players, each obtaining the utility r . However, if one of these latter player plays the diligent computation and honest submission without leaking, he will obtain $(n-1)(1-q_i)f$. If $(n-1)(1-q_i)f > \text{cost}(d) - \text{cost}(l_i) + n(\lambda)$ for all non-negligible functions $n(\cdot)$, the computational Nash equilibrium is spoiled. Again it is straightforward to deduce that $f > \frac{r}{(n-1)(1-n(\lambda))}$ for all negligible functions $n(\cdot)$ is sufficient as in the proof of Theorem 5.

Proof (of Theorem 4). Given that all the contractors outside the coalition play the diligent strategy, the utility of the coalition G from all non-collaborating diligent strategy is calculated as $|G|(\cdot r - \text{cost}(d))$. We stress that there must be at least 1 player outside that plays the diligent strategy as theorem covers the coalitions up to size $n-1$. If k people in the coalition submit a response obtained by a lazy algorithm, the coalition utility becomes upper bounded by $(|G|-k) \cdot r - (|G|-k)\text{cost}(d) + v'$, obtained by setting the costs of algorithms run by k deviant strategy players as 0 (for maximum utility), fines paid by them staying inside coalition (again, for maximum utility), and v' as the maximum utility a lazy algorithm can achieve. Clearly, all non-collaborating diligent strategy utility is greater than this one for any $0 < k \leq |G|$ as $r > \text{cost}(d)$. We highlight that obtaining a higher utility requires that some of the players do not compute the function but still submit the correct response. This may succeed either by copying an outsider's response or computing the correct Merkle hash without having the correct computation result, both occurring with negligible probability.

Proof (of Theorem 5). Given that all the contractors outside the coalition play the diligent strategy, the utility of the coalition G from all collaborating diligent strategy is calculated as $|G| \cdot r - \text{cost}(d)$. We stress that there must be at least 1 player outside that plays the diligent strategy as theorem covers the coalitions up to size $n-1$. If k contractors in the coalition submit a response obtained by a lazy algorithm and a contractors run the diligent algorithm, the coalition utility becomes upper bounded by $(|G|-k) \cdot r - a \cdot \text{cost}(d) + v'$, obtained by setting the costs of algorithms run by k deviant strategy players as 0 (for maximum utility),

fines paid by them staying inside coalition (again, for maximum utility), and v' as the maximum utility a lazy algorithm can achieve. Clearly, all collaborating diligent strategy utility is greater than this one for any $0 < k \leq |G|$. We highlight that obtaining a higher utility requires that none of the players compute the function. This may succeed either by copying an outsider's response or computing the correct Merkle hash without having the correct computation result, both occurring with negligible probability.